Product Engineer Portfolio

1. 핵심 역량

- **웹 성능 최적화**
  React SPA 마이그레이션 및 Webpack 번들 최적화를 통해 초기 로딩 속도 개선 (LCP 3.8s → 1.6s)
- **테스트 자동화**
  Storybook 기반 컴포넌트 개발 및 Vitest / Playwright를 활용한 유닛·E2E 회귀 테스트 구축
- **CI/CD 구축 & 협업**
  GitHub Actions 기반 CI/CD 파이프라인 구축 (배포 15m → 2m) 및 백엔드 인터페이스 설계 제안

2. 기술 스택

- **Frontend**
  TypeScript, React, Next.js, TanStack Query, Recoil, Tailwind CSS, shadcn/ui, MUI, Electron, Webpack
- **Backend**
  Python, Django, PostgreSQL, MariaDB, Socket.io, WebRTC
- **ETC**
  Docker, Nginx, GitHub Actions, Vitest, Playwright, MSW, Storybook

3. 경력

주식회사 퓨처드라이브 | 클라우드개발팀 | 프리랜서 | 2026.02 ~ 2026.06 (5개월)
주식회사 퓨처드라이브 | 클라우드개발팀 | 정규직 | 2022.09 ~ 2026.01 (3년 5개월)

- 팀 내 FE 전담, 기획·설계·배포까지 전 주기 참여
- FE 기술 스택 선정 및 아키텍처 의사결정 주도

4. 학력 및 자격증

- **학력:** 금오공과대학교 컴퓨터소프트웨어공학 학사 졸업 (2020.03 ~ 2022.08)
- **교육:** 패스트캠퍼스 이너써클 과정 수료 (2024.11 ~ 2025.03) · 멘토링 및 O2O 서비스 FE 개발
- **자격증:** 정보처리기사 (2021.08) | SQLD (2024.09)
- **외국어:** 영어 OPIc IH

① 공간 예약 서비스

개인 프로젝트 · 2026.05 ~ 2026.06 · 1인 개발 (기획, 디자인, FE/BE 개발, 운영)

TypeScript · **Next.js 16 (App Router)** · **React 19** · **TanStack Query** · **Storybook** · **Vitest** · **Playwright** · **Tailwind CSS v4** · Drizzle ORM · Google Calendar API

Storybook 기반 UI 독립 개발 및 검증

배경 예약 중첩, 외부 행사, 로딩 및 에러 등 캘린더 화면이 가지는 다양한 예외 상태를 백엔드 API 데이터 없이 재현하고 검증하기 어려움

작업 Storybook을 활용해 백엔드 API 의존 없이 타임라인·캘린더 컴포넌트의 다양한 상태(기본, 예약 중첩, 외부 행사 등)를 독립적으로 격리 개발하고 검증

결과 API 의존 없는 UI 독립 개발 및 레이아웃 엣지 케이스 버그 사전 방지

TanStack Query 기반 무한 스크롤 & 비동기 데이터 캐싱

배경 대량의 예약 목록 조회 시 발생하는 초기 데이터 로딩 지연 및 잦은 페이지 이동에 따른 반복적인 네트워크 요청

작업 useInfiniteQuery와 Intersection Observer를 결합해 대용량 목록 무한 스크롤 페이징을 구현하고, staleTime 캐싱 옵션을 적용해 네트워크 재요청 최적화

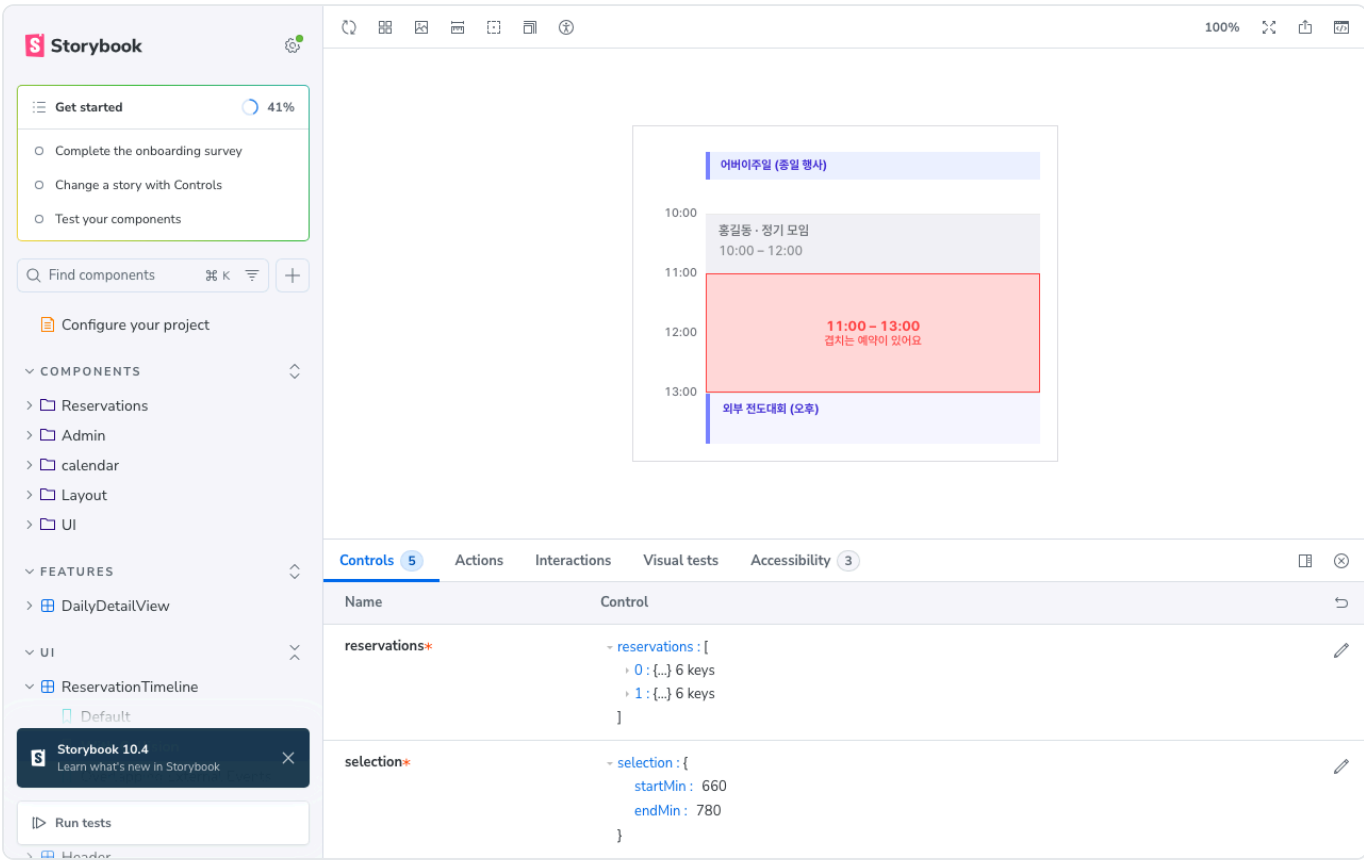
결과 불필요한 재요청을 방지하고 매끄러운 무한 스크롤 리스트 사용자 경험 구현

Vitest & Playwright 기반 핵심 로직 검증 및 E2E 테스트 자동화

배경 캘린더 일정 연산 정밀도 보장 및 회원가입~예약으로 이어지는 핵심 사용자 흐름의 회귀 에러 방지 필요성

작업 Vitest로 캘린더 내부 예약과 외부 구글 캘린더 일정의 정렬·시간 계산 함수 유닛 테스트를 작성하고, Playwright E2E 환경에서 외부 API 의존 없는 목킹(Mocking) 모드를 구축하여 전체 유저 흐름 자동화 테스트 수행

결과 핵심 계산 로직 및 사용자 서비스 여정에 대한 신뢰도 높은 회귀 테스트 안전망 구축



② 오토노미 스튜디오

(주)퓨처드라이브 · 2023.01 ~ 2024.12 · 4인 팀(FE 2인, BE 2인) · FE 아키텍처 및 CI/CD 주도
자율주행 개발 환경 설정, 모듈 실행, 로그 관리를 웹 UI로 통합한 웹 기반 개발 플랫폼

TypeScript · **React** · **Recoil** · **MUI** · **Socket.io** · **Webpack** · Django · GitHub Actions · Docker

Django 템플릿 → React 아키텍처 점진적 마이그레이션

배경 서버-UI 강결합으로 인한 확장성 한계 및 페이지 전환 시마다 전체 새로고침 발생으로 사용자 경험(UX) 저하

작업 전환 PoC로 생산성을 입증한 후 라우트 단위 점진적 마이그레이션 수행, JSON REST API 분리 및 MUI 공통 UI 컴포넌트 재사용 체계 구축

결과 화면별 중복 구현을 제거하고 독립적인 SPA 프론트엔드 아키텍처 완성

Webpack 라우트 스플리팅 & 리소스 번들 최적화

배경 단일 대형 번들 파일로 인해 서비스 접속 시 초기 LCP 로딩 속도가 3.8초까지 지연되는 문제

작업 Webpack bundle-analyzer 분석 후 React.lazy/Suspense 라우트 단위 코드 스플리팅 적용, WOFF2 폰트 및 WebP 이미지 리소스 변환

결과 초기 번들 사이즈 80% 감소 및 초기 진입 속도 **3.8초 → 1.6초 단축**

Git Flow 도입 및 GitHub Actions CI/CD 자동화

배경 브랜치 전략 부재 및 서버 직접 접속 수동 배포로 월 평균 5건의 배포 오류 발생

작업 전사 Git Flow 운영 규약을 정비하여 가이드를 작성하고, GitHub Actions 기반 자동 빌드·테스트·배포 파이프라인 구축

결과 배포 리드타임 **15분 → 2분** 단축 및 수동 배포 실패 건수 **0건 달성**

대용량 파일 스트리밍 다운로드

배경 8GB 대용량 파일 전송 시 서버 및 단말 메모리 초과(OOM) 발생 및 온프레미스 네트워크 대역폭 잠식

작업 다운로드 전용 서버 분리 및 최대 전송 대역폭 제한 설정, Django StreamingHttpResponse를 활용한 스트리밍 다운로드 환경 구축

결과 8GB 원격 전송 중 단말·서버 프로세스 강제 종료를 방지하고 관제 대역폭 유지

HTTP Polling → Socket.io 기반 양방향 실시간 통신 개편

배경 주기적 HTTP Polling으로 인한 서버 요청 낭비 및 로그/상태 데이터 동기화 지연

작업 Socket.io 양방향 웹소켓 통신 구조로 전면 개편하고, 데이터 로딩 구간 Skeleton UI 도입

결과 데이터 지연 시간을 **2초에서 0.5초로 75% 단축**하고 사용자 체감 반응성 대폭 개선

③ 충전 트레일러 관제 서비스 (FMS)

(주)퓨처드라이브 · 2025.01 ~ 2025.12 · 4인 팀(FE 1인 전담, BE 2인, 기획 1인)

TypeScript · **Next.js** · **React** · **Recoil** · **Tailwind CSS** · **Socket.io** · **MSW** · **Vitest** · Kakao Map API · Docker · Nginx

Blue-Green 배포 전환으로 배포 다운타임 제거

배경 컨테이너 재시작 방식의 배포로 매 배포마다 서비스 다운타임 발생

작업 nginx·Docker·bash 스크립트로 Blue-Green 배포 구축, 신규 컨테이너 정상 기동 확인 후 트래픽 전환 및 기존 컨테이너 제거

결과 배포 시 다운타임 없는 무중단 배포 환경 구현

MSW(Mock Service Worker) 기반 병렬 개발 환경 구축

배경 백엔드 API 개발 지연으로 프론트엔드 화면 구현 및 연동 테스트 병목 발생

작업 MSW로 네트워크 레벨 모킹 환경을 구축하여 API 미완성 상태에서도 엣지 케이스 및 예외 시나리오까지 검증

결과 백엔드와의 병렬 개발로 전체 개발 일정 **2주 단축** 및 실서버 연동 시 FE 코드 수정 최소화

실시간 관제 화면 마커 순간이동 현상 해소

배경 소켓 수신 좌표를 지도 마커에 단순 대입 시 마커 이동이 끊기거나 순간이동하듯 부자연스럽게 보이는 현상 발생

작업 Socket.io 수신 데이터를 렌더링 큐에 수집해 requestAnimationFrame 단위로 마커를 일괄 업데이트하고, 위치 좌표 보간 연산으로 부드러운 주행 궤적 렌더링 구현

결과 마커 불연속 이동을 방지하고 매끄러운 60fps 주행 궤적 시각화 구현

충전사·주행방식 조합별 예약 절차 조건분기 제거

배경 충전사나 주행방식이 추가될 때마다 그 조합 수만큼 if-else 분기가 늘어날 위험 존재

작업 조합 문자열을 키로, 예약 액션을 enum으로 정의한 매핑 테이블 구조를 도입해 조합별 절차를 정의

결과 신규 조합을 매핑 테이블에 추가하는 것만으로 확장 가능한 구조 확립



④ 차량용 원격데스크톱 솔루션

(주)퓨처드라이브 · 2024.01 ~ 2024.12 · 3인 팀(FE·BE 1인 전담, 테스트 2인)

TypeScript · **React** · **WebRTC** · **Socket.io** · GStreamer · Django · Docker

WebRTC Data Channel을 통한 제어 지연 단축

배경 기존 WebSocket 기반 입력 이벤트 전송 구조에서 네트워크 패킷 처리 오버헤드로 약 500ms의 제어 반응 지연 발생

작업 WebRTC Data Channel을 도입해 P2P 제어 전송 경로로 개편하고, Signaling 서버 구현 및 STUN/TURN 서버 환경 구축 후 18개 시나리오 실측 검증

결과 제어 명령 전송 지연 **500ms → 50ms (90% 단축)**

포인터(mousemove) 이벤트 샘플링 처리

배경 초당 수백 회 발생하는 mousemove 포인터 이벤트로 차량 단말 CPU 연산 부하 및 이벤트 유실 발생

작업 송신단에서 10:1 이벤트 샘플링 필터를 구현해 전송 빈도를 초당 입력의 1/10 수준으로 감소

결과 차량 단말 CPU 연산 부하를 줄여 입력 이벤트 누락 방지

WebRTC 끊김 감지 및 자동 재접속·재협상

배경 무선 네트워크 변동(5G/WiFi 음영)으로 스트리밍 단절 시 운영자의 수동 재접속 번거로움 발생

작업 서버 측 GStreamer 프레임 송출 모니터링과 함께, WebRTC 연결 상태와 실제 영상 재생 상태를 함께 검사하는 이중 감지 로직을 구현하고, 단절 시 자동으로 재접속 또는 재협상 수행

결과 네트워크 유실 시 수초 내 수동 개입 없이 자동 복구되는 안정적인 스트리밍 환경 구현

다중 사용자 스트리밍 리소스 최적화

배경 다중 사용자 접속 시 화면 캡처가 사용자 수만큼 중복 실행되어 리소스 낭비 위험 존재

작업 캡처·인코딩은 tee로 공유하고 사용자별 webrtcbin만 분기하는 구조로 설계, 활성 연결을 Map으로 추적해 전원 종료 시 파이프라인을 PAUSED로 전환

결과 캡처 리소스를 사용자 수와 무관하게 1회만 사용하고, 유휴 상태 시 자동 정지로 불필요한 리소스 소모 제거

